

Projekt im Master Informatik: Datenbanken und Informationssysteme

Jean Baptiste Landry Mbem, Matricule: ~~581455~~
Nelson Aurelien Todem Fotso, Matricule: ~~141414~~

August 2024

Abstract

The problem addressed in this project is the complexity involved in managing and querying graph databases, particularly Neo4j, which often requires a high level of technical expertise. This complexity can be a significant barrier for non-technical users who need to interact with graph data, such as in creating nodes, establishing relationships, or querying the database. The proposed solution is a Python-based application with a Graphical User Interface (GUI) that simplifies these tasks by allowing users to create and manage graph databases directly from Comma Separated Value (CSV) files. The application introduces role-based access control, distinguishing between administrators who can modify the database and regular users who can query it. This solution is superior because it democratizes access to graph database technology, enabling users with minimal technical knowledge to manage complex data relationships effectively, while ensuring data integrity and security through controlled access.

0.1 Introduction

Graph databases, particularly Neo4j, have become increasingly popular due to their ability to efficiently manage and query data with complex relationships. Unlike traditional relational databases, graph databases are optimized for traversing relationships, making them ideal for applications such as social networks, recommendation engines, and streaming apps. However, interacting with graph databases typically requires a certain level of technical expertise, which can be a barrier for many users. This project addresses this challenge by developing a Python-based application with a GUI that allows users to create and manage graph databases from simple CSV files. The chosen domain is the field of Medicine involving Doctors, Patients, Drugs, Meeting between Doctors and Patients, Pharmacies handling Drugs, and main the main disease handled through out this project is Diabetes. The GUI is designed to cater to both administrators, who have full control over the database, and regular users, who can query and analyze the data.

0.2 Related Work

The use of graph databases has expanded in various fields due to their efficiency in handling connected data. Tools such as Neo4j Desktop and Cypher Shell provide interfaces for interacting with graph databases, but they are often geared toward users with advanced technical skills[1]. Previous efforts to simplify database management include various web-based and desktop applications, but these often lack the flexibility or user-friendliness required for widespread adoption. This project builds on these efforts by introducing a GUI that simplifies database creation and management tasks, making graph database technology more accessible to a broader audience.

0.3 Methodology

0.3.1 System Architecture

The application is structured around two primary user roles: admin and user. The admin role allows for full database management, including the creation of nodes and relationships, while the user role is restricted to querying the database and viewing results. The system is implemented using Python's Tkinter library for the GUI, and the Neo4j Python driver for interacting with the Neo4j database.

0.3.2 System Modelling

The presented application was tailored based on UML Class Diagram to depict the overall system architecture including Nodes and their attributes as well as the connections between these nodes. This provided a general view of the application and greatly help during the implementation. The database presented here represents a system in which it is possible to create and retrieve data relating to the disease known as Diabetes. The data processed represents only a sample of what could exist on a global scale. Diabetes is a disease that is present on all continents, in our case we have chosen 5, in each of these continents we have randomly chosen 5 countries. In each of these countries there is a Ministry of Health, which has a department responsible for hospitals. Each of these hospitals has a diabetes department employing a number of doctors. Each of the doctors looks after 2 patients for whom they can prescribe medication. Medicines that the patient can find in a given pharmacy.

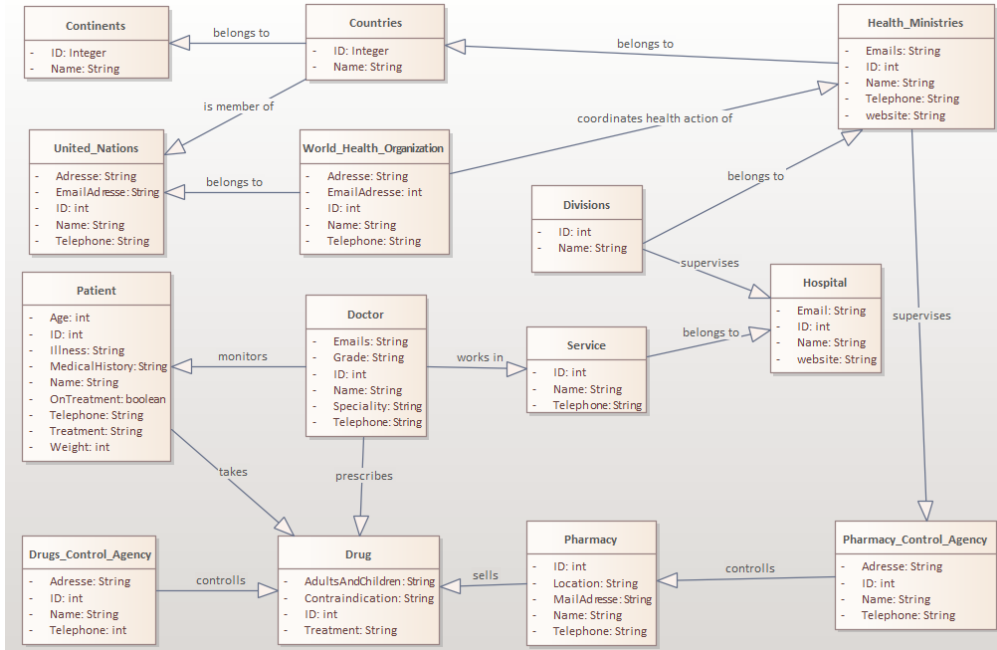


Figure 1: System General Overview displaying all nodes, their attributes and the relationship between them

0.3.3 Application Design

Upon launching the application, users are presented with a login page where they must enter a username, password, and select their role (admin or user). The username and password are used to establish a connection to the Neo4j database, allowing for personalized access based on the user's credentials.

- **Admin Interface:** Admins can import CSV files to create nodes and relationships in the database. The nodes are created based on the labels provided by the user, with each row in the CSV representing a node and each column representing a node property. Relationships between nodes can also be defined using CSV files, where specific columns represent the identifiers of the start and end nodes. Additional features include the ability to delete nodes and relationships, execute Cypher queries, and print query results.
- **User Interface:** Users are provided with a simplified interface where they can execute queries and view results. Unlike admins, users do not have the ability to modify the database structure. This restriction is designed to prevent unauthorized changes and ensure data integrity.

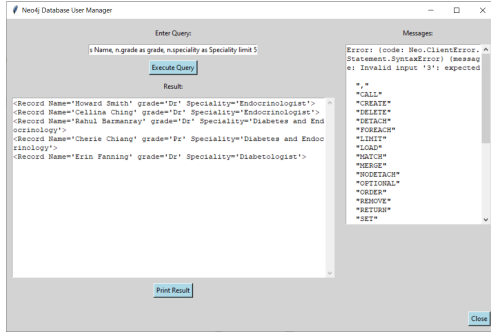


Figure 2: User Interface

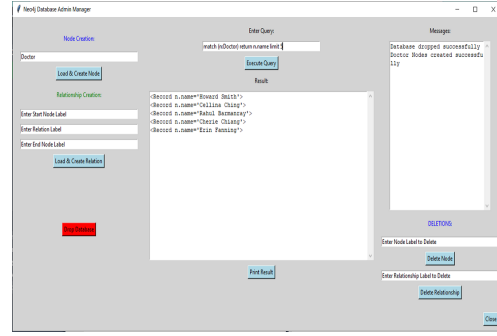


Figure 3: Admin Interface

0.4 Implementation

0.4.1 Functional Requirements

The application consists of several key functions that handle the core operations related to database management and interaction:

1. **"create-node" Function:** This function is responsible for creating new node instances in the Neo4j database automatically. It takes a label and a dictionary of attributes (derived from the CSV file) and constructs a Cypher query to insert the node into the database. The user does so by entering the node label in the input button and then uploading the corresponding CSV file, see Figure 4.

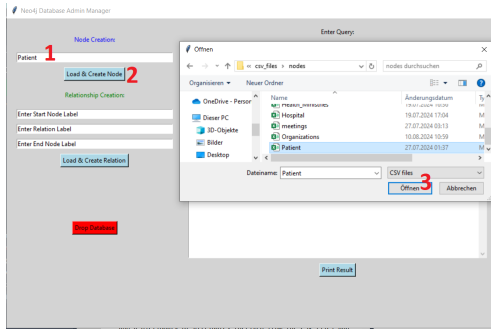


Figure 4: Node Creation Steps

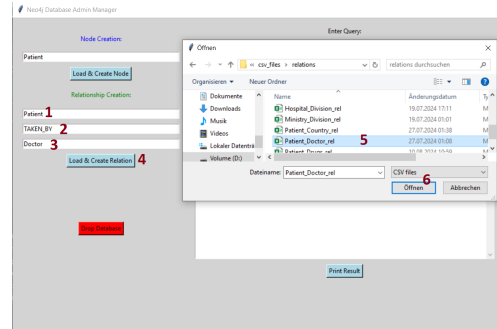


Figure 5: Relationship Creation Steps

2. **"create-relationship" Function:** This function manages the creation of relationships between nodes. It uses a CSV file where specific columns denote the start node ID, end node ID, and the relationship name, see Figure 5. The function matches the nodes based on their IDs and establishes a relationship between them.
3. **"load-data" Function:** This function reads the CSV files and converts the data into a python dictionary for insertion into the Neo4j database. It supports both "," and ";" delimiters, allowing flexibility in the input file format. It is associated with the create node and create relation functions. In the create node, the order of the columns does not matter whereas this is not the case in the create relation function. Here, the first column represents

the first node and must match with the first node label when creating the relation in the admin GUI. The second column represents the second node.

```

72 # Load CSV data
73 @context
74 def load_data(file_path):
75     data = []
76     with open(file_path, 'r') as file:
77         first_line = file.readline()
78         delimiter = ',' if ',' in first_line else ';'
79         file.seek(0)
80         csv_reader = csv.DictReader(file, delimiter=delimiter)
81         for row in csv_reader:
82             data.append(row)
83     return data
84
85 # Login Page
86 @context
87 class LoginPage:
88
89 # Admin Interface
90 @context
91 class AdminUI:

```

Figure 6: Load CSV File

```

92 def load_data(file_path):
93     file_path = pathlib.Path(file_path).as_posix()
94     if not file_path:
95         return
96
97     data = load_data(file_path)
98     label = self.node_label_entry.get()
99
100     if not label:
101         messagebox.showerror("Error", "Node label cannot be empty.")
102         return
103
104     db = Neo4jDatabase(self.url, self.user, self.password)
105     try:
106         # we add constraints: ID uniqueness on the ID of each node
107         db.create_schema(label)
108         for row in data:
109             db.create_node(label, row)
110
111     db.close()
112     messagebox.showinfo("Success", f"Label Nodes created successfully from {file_path}")
113     self.message_text.insert(tk.END, f"Label Nodes created successfully\n")
114 except Exception as e:
115     self.message_text.insert(tk.END, f"Error: {str(e)}\n")
116 db.close()

```

Figure 7: Load Data from File

4. **"execute-query" Function:** This function allows users to execute arbitrary Cypher queries. Admins can use it to run complex queries to the database, while users can use it to retrieve and view data.
5. **"drop-database" Function:** Exclusively available to admins, this function deletes all nodes and relationships in the database, effectively resetting it.

These functions work together to provide a seamless experience for users, whether they are creating and managing graph data or simply querying the database.

0.4.2 CSV File Formats and Their Roles in the Application

In the context of this project, CSV (Comma-Separated Values) files play a crucial role in defining both the nodes and relationships within the graph database. These files are utilized to import data into the Neo4j graph database, allowing users to quickly and efficiently create complex networks of interconnected nodes. This section provides a detailed explanation of the specific CSV formats used for nodes and relationships, and how they interact with the application.

CSV Files for Nodes

- **Purpose:** CSV files for nodes are used to define the entities or objects that will populate the graph database. Each row in a CSV file corresponds to a single node, and the columns represent the properties of that node
- **Structure:** A typical CSV file for nodes in this project might look like in the figure below:
- **Components:** The first row/line of the CSV file also called **Header Row** contains the names of the properties that will be associated with each node. In the example above, the properties include **id**, **name**, **grade**, **speciality**, **email**, **contact**.

Subsequent rows called **Data Row** represent individual nodes, with each column containing the value for the corresponding property. For example, the first row after the header defines a node with id 2, named "Jean Claude Nganou Mbany", who is a "Senior Surgeon" specialized in "Diabetology and Cardiology".

- **Operation in Code:** When a user imports a CSV file for nodes, the application reads the file, extracts the data, and creates nodes in the Neo4j database. Each node is labeled according to the user's input, and the properties are assigned based on the columns in the

CSV file. The id field is typically used as a unique identifier for each node and a property constraint is set on this id as Unique.

CSV Files for Relationships

- **Purpose:** CSV files for relationships define the connections between nodes in the graph. Each row/line represents a relationship between two nodes previously created
- **Structure:** A typical CSV file for nodes might look like in the figure below:

- **Components:** The first row contains the names of the fields that will be used to establish the relationships between nodes. In the example above, the fields are: **start-id, end-id**.

Each following row describes a relationship between two nodes identified by start-id and end-id (e.g., The relation: "WORKS-AT" between Doctor and Hospital based on their respective IDs).

- **Operation in Code:** When importing a relationship CSV file, the application reads the data and matches the start-id and end-id with the corresponding nodes in the database. It then creates a relationship of the specified type between these nodes and assigns any additional properties from the file to this relationship. The code ensures that both nodes exist before attempting to create a relationship.

NOTE that order of the columns in these CSV files determines the direction of the relation. In other words, **If the first column of the CSV file represent IDs of Doctors and the second column those of Hospital for the relation WORKS-AT, then when creating this relation in the application console, this node order should be maintain: Doctor - WORKS-AT ->Hospital.**

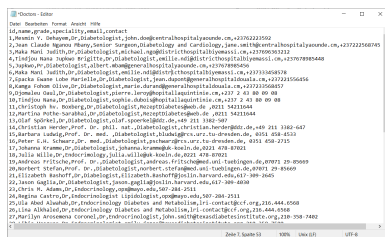


Figure 8: Doctor CSV File Format

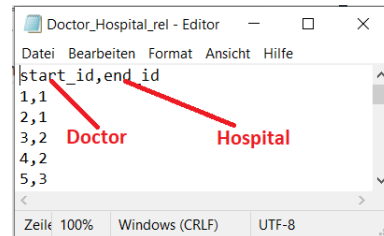


Figure 9: WORKS-AT Relationship CSV File Format

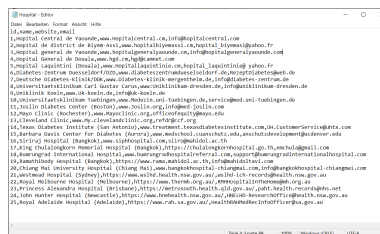


Figure 10: Hospital CSV File Format

0.5 Results and Discussion

The application was tested in a variety of scenarios to ensure its robustness and usability. Admins were able to successfully create nodes and relationships from CSV files, manage database schema constraints, and execute complex queries. The user interface allowed non-technical users to query the database with ease, demonstrating the effectiveness of the role-based access control. The GUI's responsiveness was validated across different devices, ensuring a consistent user experience.

0.6 Conclusion

This project successfully developed a Python-based application that simplifies the creation and management of Neo4j graph databases from CSV files. By incorporating a user-friendly GUI, the application makes graph database technology accessible to a broader audience, including those with limited technical expertise. The role-based access control ensures that only authorized users can make critical changes to the database, while still allowing regular users to explore and analyze the data. Future work could involve extending the application to support additional graph database features, such as advanced data visualization and graph algorithms.

Bibliography

- [1] Robinson, I., Webber, J., Eifrem, E. (2015). Graph Databases: New Opportunities for Connected Data. O'Reilly Media, Inc.